Project - High Level Design

On

Deploy Manufacturing App to Kubernetes

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

S. NO	NAME	ENROLLMENT NO.
1.	**Anurag Didolkar**	**EN22CS301169**
2.	**Amit Patidar**	**EN22CS301114**
3.	**Arsh Patidar**	**EN22CS301204**
4.	**Anuj Singh Rathore**	**EN22CS301166**
5.	**Avani Gupta**	**EN22CS301236**
6.	**Aniket Kushwah**	**EN22CS301124**

Group Name: 01D6

Project Number: AAI-12

Industry Mentor Name: Vaibhav

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-2026

Table of Contents

1. Introduction.

    1.1. Scope of the document. 1.2. Intended Audience
    1.3. System overview.

2. System Design.

    2.1. Application Design 2.2. Process Flow.
    2.3. Information Flow.
    2.4. Components Design 2.5.
    Key Design Considerations
    2.6. API Catalogue.

3. Interfaces

4. State and Session Management

5. Caching

6. Functional Requirements

    6.1. User Authentication & Authorization
    6.2. Infrastructure Configuration Management
    6.3. Provisioning Control & Monitoring
    6.4. Dockerfile (Application Containerization)
    6.5. GitHub Actions (Workflow Configuration)
    6.6. CI/CD Strategy (Continuous Integration & Deployment)

7. Non-Functional Requirements

8. References

9. GitHub

1. Introduction

This document presents the system design for the **Deploy Node.js Retail App to Kubernetes** project, developed as part of the Datagami Skill Based Course at Medicaps University.

The project demonstrates a complete DevOps implementation involving containerization, cloud-native orchestration, and automated deployment of a **Node.js Retail Application** using AWS EKS.

The DevOps infrastructure ensures the application is reliably packaged, deployed, and updated on a production-grade AWS EKS Kubernetes cluster with **zero downtime on every release**.

1.1 Scope of the Document This document covers:

- System architecture and component interactions
- Docker containerization strategy for Node.js app
- Kubernetes deployment and service configuration
- CI/CD pipeline using GitHub Actions
- AWS EKS infrastructure setup
- Secrets and environment variable handling
- Security and design decisions
- API catalogue (health + retail endpoints)

System Architecture:

1. High-Level Overview

The architecture represents a **fully automated CI/CD pipeline** that deploys a **Node.js Retail Application** to an **AWS EKS (Kubernetes) cluster** with **zero-downtime rolling updates**.

The system follows this flow:

```
Developer → GitHub → GitHub Actions → Docker Hub → AWS EKS →
Load Balancer → Users
```

2. Key Components Explained

1. Developer (Code Source)

- Developers write and update the Node.js application.
- A simple **`git push`** triggers the entire pipeline.
- No manual deployment is required.

2. GitHub Actions (CI/CD Engine)

This is the **automation brain** of the system.

It performs:

1. **Checkout Code**
2. **Build Docker Image**
3. **Push Image to Docker Hub**
4. **Authenticate with AWS**
5. **Deploy to Kubernetes**

This ensures **continuous integration + continuous deployment (CI/CD)**. **3. Docker Hub (Container Registry)**

- Stores the built Docker images.
- Acts as a **central image repository**.
- Kubernetes pulls images from here during deployment.

4. AWS EKS (Kubernetes Cluster)

This is where your application actually runs.

Inside EKS:

a. Kubernetes Deployment

- Manages application pods.
- Maintains:
  - `replicas: 2` (high availability)
  - Rolling update strategy

b. Node.js Pods

Each pod runs:

- Node.js + Express app
- Port **3000**

Example:

```
Node.js Pod v1 → Old version
Node.js Pod v2 → New version
```

c. Auto Scaling EC2 Nodes

- EKS runs on EC2 instances.
- Automatically scales based on load.
- Ensures:
  - Performance
  - Cost efficiency

5. AWS Load Balancer

- Exposes the app to the internet.
- Routes traffic to healthy pods only.

- Runs on **Port 80 (HTTP)**.

6. End Users

- Access application via:
- Requests flow through Load Balancer → Kubernetes → Pods.

3. Zero-Downtime Rolling Update (Core Concept)

This is the **most important feature** of your architecture.

How it works:

1. New version (v2) pod is created
2. Health check is performed
3. Traffic is shifted gradually
4. Old version (v1) pod is terminated

Configuration:

```
replicas:                                                    2
maxSurge:                                                    1
maxUnavailable: 0
```

Result:

- Application **never goes down**
- Users experience **no interruption**

4. Request Flow (Runtime)

```
User                                                    Browser
   ↓
```

```
AWS Load Balancer (Port 80)
↓
Kubernetes Service
↓
Node.js Pod (Port 3000)
↓
Application Response
```

5. CI/CD Flow (Deployment)

```
Git Push
↓
GitHub Actions
↓
Build Docker Image
↓
Push to Docker Hub
↓
Deploy to EKS
↓
Rolling Update
↓
App Live
```

6. Key Architecture Benefits

High Availability

- Multiple replicas ensure app is always running

Zero Downtime

- Rolling updates prevent service interruption

Scalability

- EKS auto-scales nodes and pods

Automation

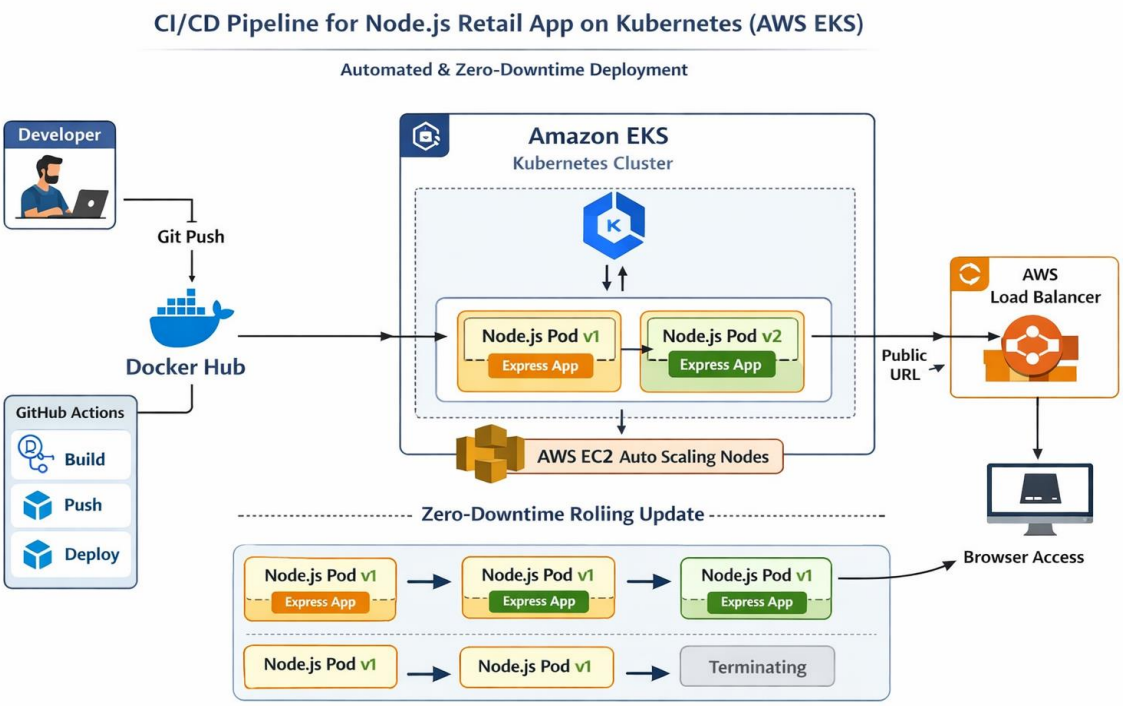
- Fully automated pipeline (no manual deployment)

Security

- Secrets stored in GitHub Secrets (not in code)

Cost Efficiency

- Uses minimal EC2 instances (t3.small)



1.2 Intended Audience

Audience	Purpose
DevOps Engineers	Understand CI/CD pipeline & infra
Backend Developers	Understand containerization
Cloud Architects	Review AWS EKS design
Evaluators	Assess implementation
Maintainers	Extend system

Project Mentors	Review the overall system design for guidance and feedback
Future Maintainers	Onboard quickly and extend the infrastructure

A basic understanding of Docker, Kubernetes, GitHub Actions, and AWS services is recommended for reading this document.

1.3 System Overview

The system can follow:

Option 1: Single Repository

- Contains Node.js app + DevOps

Option 2: Two-Repository (Recommended)

- Repo 1 → Node.js Retail App
- Repo 2 → DevOps Infrastructure

Pipeline flow:

```
Git Push → GitHub Actions → Docker Build → Docker Hub → EKS Deployment → Load Balancer → Users
```

Kubernetes performs **Zero-Downtime Rolling Updates**, ensuring continuous availability.

2. System Design

2.1 Application Design

Architecture: Two-Tier

Frontend Layer

- Can be:
  - EJS / React / Static HTML
- Runs via Express server

Backend Layer

- Node.js + Express API

Example Structure

```
src/
 ├── routes/
 ├── controllers/
 ├── models/
 └── app.js
```

Container Design

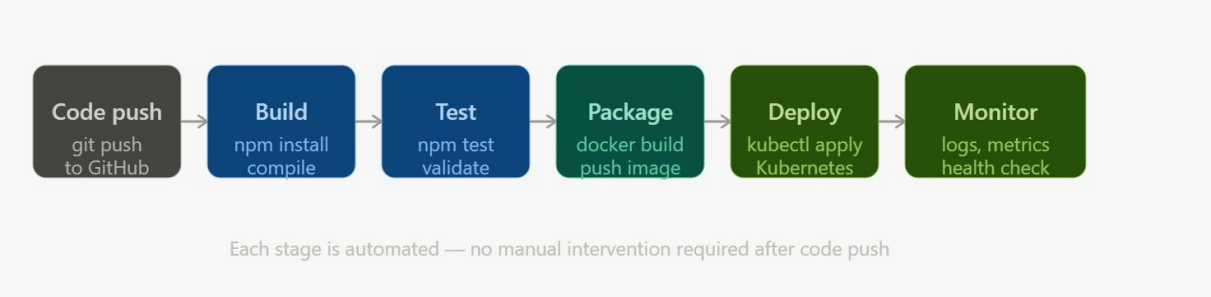
Property	Value
Base Image	node:18-alpine
Port	3000
Entry	node server.js

Steps:

- Copy package.json
- npm install
- Copy source
- Start server

2.2 Process Flow

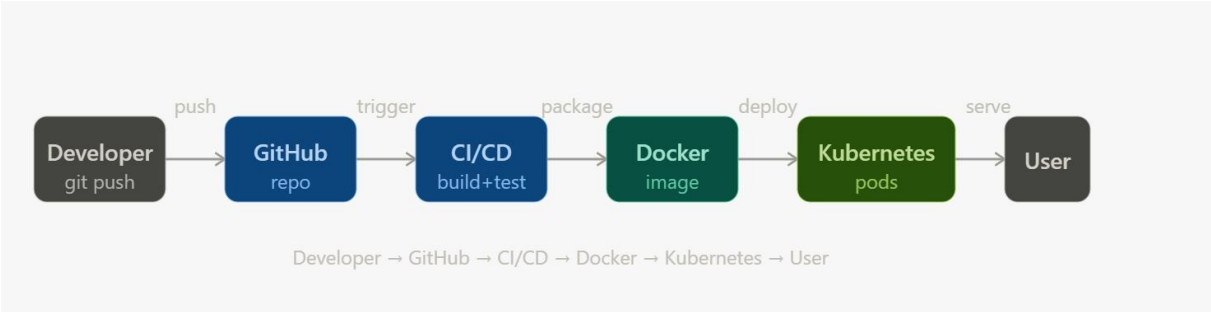
The end-to-end process flow from code change to live deployment is as follows:



2.3 Information Flow

The information flow describes how data moves through the system at runtime, from the user's browser to the AI agents and back.

User Request Flow:



2.4 Components Design

Component 1- Dockerfile

Property	Value
Base Image	node:18-alpine
Port	3000
Command	node server.js

Key steps: install system dependencies → copy requirements.txt → clone AI repo → install Python packages → create artifacts directory → set environment variables → define health check → set CMD.

Component 2- Kubernetes Deployment (deployment.yaml)

Parameter	Value
replicas	2
strategy	RollingUpdate
maxSurge	1
maxUnavailable	0
containerPort	3000

Component 3- Kubernetes Service

Parameter	Value
type	LoadBalancer
port	80
targetPort	3000

Component 4- GitHub Actions Pipeline

Step	Action
1	Checkout
2	Docker Login
3	Build Image
4	Push Image
5	AWS Auth
6	Deploy
7	Verify

Component 5- AWS Infrastructure

Component	Config
EKS	ap-south-1
Nodes	t3.small
Registry	Docker Hub
Load Balancer	AWS ELB

2.5 Key Design Considerations

1.

   a **Zero Downtime Deployment**

2. `maxUnavailable: 0`
   a **Health Checks**
3. Node.js / endpoint
   a **Security**
4. Secrets stored in GitHub
   a **Scalability**
5. EKS auto-scaling
   a **Cost Efficiency**
6. t3.small instances

2.6 API Catalogue

Although this is a DevOps project with no custom REST API, the following endpoints are relevant to the system's operation:

Internal Health Endpoints (Streamlit Built-in)

Endpoint	Method	Purpose
/	GET	Health check

Response:

HTTP 200 OK

Body: "ok"

External API Used by Application

API	Purpose
Payment API	Checkout
Product API	Inventory

Docker Hub Registry API (used by GitHub Actions)

Action	Command	Purpose
Push Image	docker push tiwariom/multi-agentmanufacturing:latest	Upload built image to registry
Pull Image	Handled by EKS automatically	EKS pulls image on pod creation

AWS APIs (used by GitHub Actions)

API	Command	Purpose
EKS kubeconfig	aws eks updatekubeconfig	Authenticate kubectl with EKS cluster
Kubernetes API	kubectl apply -f deployment.yaml	Create/update deployment on cluster
Kubernetes API	kubectl apply -f service.yaml	Create/update LoadBalancer service
Kubernetes API	kubectl rollout status	Verify deployment completed successfully

3 Interfaces

- User Interface → Browser
- DevOps Interface → GitHub, AWS, Docker
- System Flow →

```
Browser → LoadBalancer → Service → Pod
```

4 State and Session Management

- Stateless Node.js containers
- Session handled via:
- JWT / cookies (optional)

.

5 Caching

The Dockerfile is structured to maximize Docker's layer cache. System dependencies and pip packages are installed before the git clone step so if only the AI repo code changes, Docker reuses all cached layers and skips straight to cloning. This reduces rebuild time significantly. imagePullPolicy: Always is set in the deployment manifest to ensure Kubernetes always pulls the latest image on every deploy, bypassing node-level cache. No application-level caching is implemented every agent run makes a fresh live call to the Gemini LLM API, which is intentional as supplier data must always be current.

6 Functional Requirements

6.1 User Authentication and Authorization

- Optional login system

6.2 Infrastructure Configuration Management

- Git-based version control

6.3 Provisioning Control and Monitoring

- Kubernetes probes

6.4 Dockerfile (Application Containerization)

- Uses Node.js base image
- Installs dependencies

6.5 GitHub Actions (Workflow Configuration)

- Auto trigger on push
- Fully automated deployment

6.6 CI/CD Strategy

- Every push triggers deployment
- Rolling update ensures no downtime

7 Non-Functional Requirements

- **Availability** → Always running (2 replicas)
- **Scalability** → Auto scaling
- **Security** → Secrets management
- **Performance** → Low latency (Mumbai region)
- **Maintainability** → Automated CI/CD
- **Cost Efficiency** → Low-cost EC2

8 References

- The following resources were referred to during the development and implementation of the project:

- • Official Node.js Documentation – https://nodejs.org
- • Docker Documentation – https://docs.docker.com
- • Kubernetes Documentation – https://kubernetes.io/docs
- • Git Documentation – https://git-scm.com/docs
- • GitHub Documentation – https://docs.github.com
- • GitHub Actions Documentation – https://docs.github.com/en/actions

9 GitHub

- https://github.com/ARSH2103/CI-CD-for-Node.js-Retail-App-.git